6 - Sort

Sorting means to structure the elements in a set with a specific order according to a sorting criteria which we may refer to as an ordered set. Data in arrays has a natural **positional order** corresponding to index due to the nature of the arrays; however, we often want to impose an additional order that allows and even utilizes the cross-referencing of positional order with a very specific order that facilitates more efficient searching beyond brute force iteration.

> Positional Order: Computers really cannot model the abstract set without some effort due to the structuring imposed by memory. Arrays are just contiguous segments of memory and are offset indexed from 0 to $n-1$ based on the starting address of the array. This means that a positional order is always imposed on any data inserted into an array. Whether further ordering is imposed is optional, but the positional order is an implicit feature in all arrays and can be useful feature. In fact, we leveraged the relationship between positional ordering and lexicographic ordering in binary search to be able to reduce search efficiency from $O(n)$ for data with positional order only using linear search to $O(\log_2 n)$ for data with both positional order and lexical order using binary search

Numerical data can be ordered from smallest value to largest value in **numerical order**. However, numerical order is not enough to fully specify the ordering of data in the set because data can be structured in **ascending order** or **descending order**. It therefore becomes very important to completely specify the criteria used to order data, *e.g.* ascending numerical order or descending numerical order.

Data composed of symbolic sets like strings may be sorted using a **lexicographic**, **lexical, or dictionary order**, *i.e.* ordering based on sequences of symbols like the alphabet. Likewise, lexicographic order can be ascending or descending; however, alphabetical order probably suggests ascending order without further specification while an odd ordering like reverse alphabetical order would be expected to be explicitly specified.

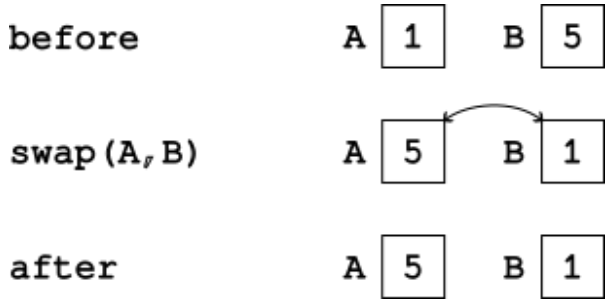
Complex data structures like objects can be ordered as well, but the sorting criteria used will depend greatly on the nature of the data stored in the object and this sorting criteria may even be variable over the lifespan of the object. Consider that student data could be ordered by name or by student id or by a combination of the two. Why a combination of the two? Consider that names are not unique so to generate a highly consistent sort structure, the name is the primary order element but if two students have the same name their records are further sorted in a secondary order by student id. We might generally refer to the ordering of objects as a **composite order** or **custom order**.

Complex data structures raise a number of questions about the implementation including how to implement simple comparison operations like less than, greater than, and equal to that are necessary to determine order.

In this section, we will explore different strategies for sorting that have been formalized into algorithms over the last century and we will analyze their efficiencies in a number of different ways. However, before we dive into different sort strategies, we first have to understand that sorting is built on one of the most basic yet fundamental algorithms for ordered sets, swap.

Swap

To swap is to exchange two elements. Given the interface `swap(A,B)`, the intent of executing the swap algorithm is to exchange the value stored in `A` with the value stored in `B` such that when the algorithm completes the new value in `A` is the value that was originally stored in `B` and the new value in `B` is the value that was originally stored in `A` before the swap was initiated. For example, suppose `A=1` and `B=5` and `swap` is performed on `(A,B)`, the result will be `A=5` and `B=1`.



Conceptually this should be a straightforward algorithm; however, there are a number of problems that may arise when implementing it in a specific language. There are at least two problems if we tried to directly implement the above process in Java.

Nuances of swap in Java

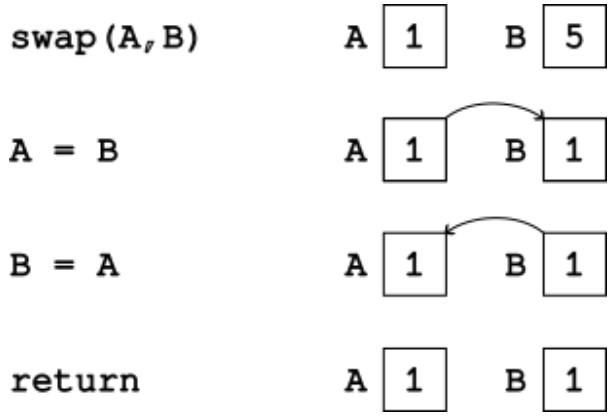
Consider the following swap operation implemented in Java:

```
int A = 1;
int B = 5;

// swap(A,B)
A = B;
B = A;

// end of swap
```

Why does the above code fail to correctly perform the swap operation?

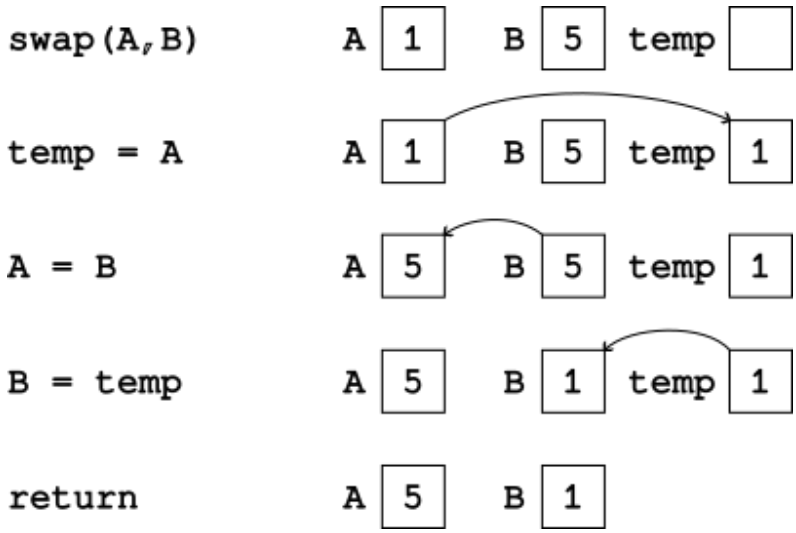


The problem is that the original value of A will be overwritten and effectively destroyed by the `A=B` statement. The original information in `A` is lost in the process, so when the `B=A` statement is reached, no net change occurs because `B` is being overwritten with its original value. How can we resolve this issue so that swap is carried out successfully? Consider the following adjustment to the implementation:

```
int A = 1;
int B = 5;
int temp;

// swap(A,B)
temp = A;
A = B;
B = temp;

// end of swap
```



The introduction of the temporary variable `temp` resolves the issue as now the original value of `A` is preserved in `temp` and even though `A` is overwritten in the `A=B` step, the original value of `A` is available in `temp` which is why it is assigned to `B` in the step before returning from `swap`.

The second issue with `swap` is a consequence of the relationship between local variables and the stack. When a function is called, the input arguments are copied to the function's parameters which is a form of protection called **pass-by-value**. In other words, the Java policy for passing arguments to functions is to pass-by-value which means that manipulation of the variable inside the function results in manipulation of a copy of the argument.

Whether or not changes to data in the function is reflected after returning from the function will depend on the metatype of the parameter. If the parameter is a primitive type, the parameter is always deep copied in Java when the function was called, so any changes to that parameter inside the function will be completely discarded when the function returns. If the parameter is a reference type, a shallow copy was made when the function was called, so any changes to the reference itself are discarded when the function returns but any changes to the parameter's referenced object are permanently changed. Consider the following example:

```
void swap(int x, int y) {
    int temp = x;
    x = y;
    y = temp;
}
```

Because `x` and `y` are input parameters, `x` and `y` are also local variables allocated on the stack. When the end of the function is reached, the stack frame containing these instances of `x` and `y` are forever removed from the stack and are lost. Stack memory for a given function only lasts until the function returns, so the results of this function are transient and lost at function return.

Alternatively, consider the following Java implementation:

```
void swap(int[] A, int i, int j) {
    int temp = A[i];
    A[i] = A[j];
    A[j] = temp;
}
```

Input parameters `i` and `j` represent the indices of the elements in array `A` that are to be swapped. Assuming that `i` and `j` are within the legal bounds of `A` and `A` is a legal reference, the implementation will swap the value in the `i`-th position with the value in the `j`-th position of `A`. This change will be a permanent change because `A` is a reference to an object on the heap where the exchange is actually carried out. Heap memory does not delete an object until there are no references remaining to that object and the calling region of code must pass in a reference to an integer array, so the change will be accessible in the calling region of code.

Stability

Not covered at this time and not a topic for quizzes. This note is to introduce this idea in future course content and to flesh out this section. For CS majors, you are encouraged to look up sort algorithm stability for a comprehensive understanding.

A simple summary of stability is to maintain order among like elements. If there are two 10's in a set, the first 10 should remain the first 10 even after sorting. A stable sorting algorithm guarantees that this order is preserved while an unstable algorithm does not.

Sorting in Place

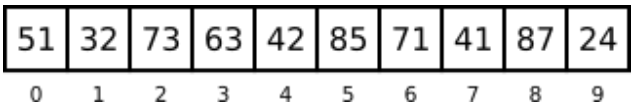
When sorting, we are really moving data from an unsorted set into a newly sorted set. If performing sort on an array, this might lead us to introduce an additional array and copy data between the unsorted array and the newly sorted array. For reasons hinted at in the discussion of pass-by-value (beyond simple considerations of space efficiency), it is often not practical to use a new array and sorting needs to instead be done as efficiently as possible in the original array. Sorting the original array is called **sorting in-place**. Sorting in-place alters the original data but this is often fine because replicating the same data twice is redundant. if two copies are necessary, the original array can still be deep copied and the copy can be sorted in-place. To identify the two sets

during the sorting processes, this text will use terminology and graphics to discriminate between the sorted and unsorted sets in one array.

Selection Sort

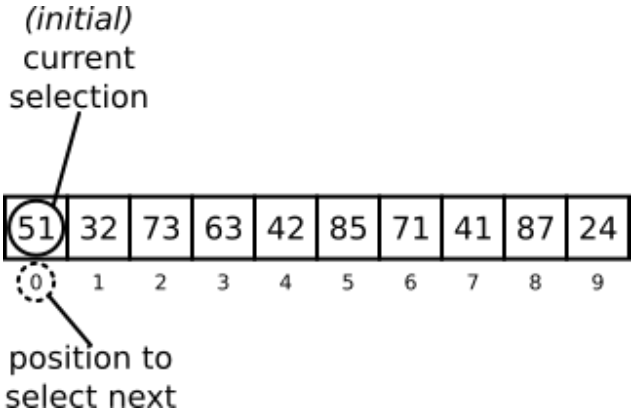
Selection Sort is arguably the most straightforward and natural strategy people typically use to sort data. It is based on the process of locating the smallest value (assuming the sort is ascending) and placing that value in the smallest position. The process is then repeated on the remaining elements which results in the second smallest element placed in the second smallest position. There are two critical flaws with this process: it is not possible to determine whether or not an element is the smallest element until it has been compared to every other element and only one element can objectively be known to be sorted with each pass through the data.

Given an array with the following data:



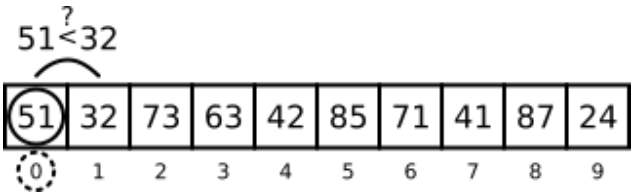
This initial array represents a completely unsorted set on which we wish to perform Selection Sort. Suppose this array is to be sorted in ascending numerical order.

Selection Sort makes an initial selection of the first value in the first position. Independently, the selected value is tracked by position in the array and the position that is being selected for is also tracked.

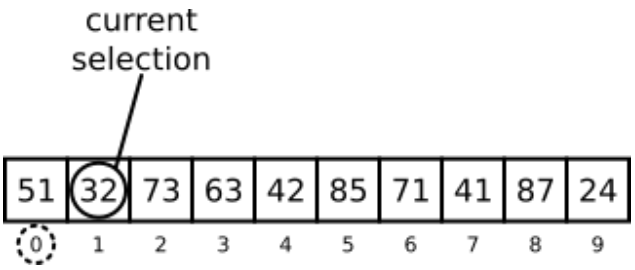


In the initial pass, selection sort scans (iterates) through all values and compares the next candidate value with the selected value. If the candidate value is less than (this specific comparison is used because sorting into ascending order) the selected value, the candidate value becomes the new selected value and the position of the new selected value is stored for future comparisons and a potential swap once the pass is completed.

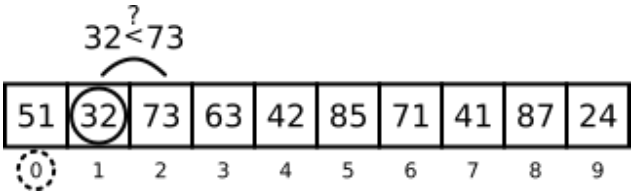
In the example, the first comparison asks if 51 is less than 32 as 32 sits in the first position of the remaining unsorted set:



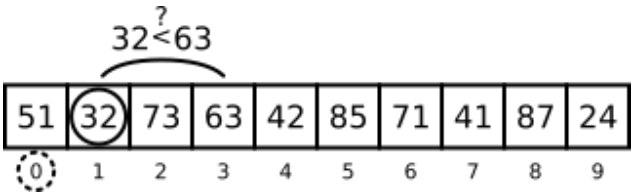
Because 32 is less than 51, 51 cannot be the smallest element in the unsorted set and so 32 replaces 51 as the current selection. Specifically, the selection algorithm starts tracking index 1 as the location of the smallest element. Any swap will be performed only once all elements in the unsorted set have been compared. The goal of the pass is to select the smallest element among the unsorted set and a separate step after the pass will swap the value into its correct position.



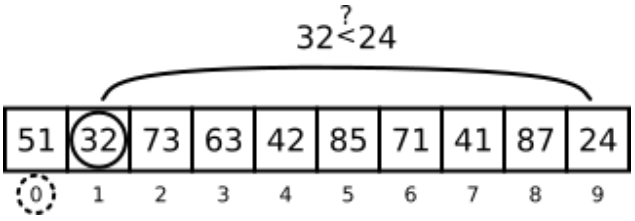
Selection Sort will continue its initial pass by comparing the new current selected value with the next candidate. In this case the next candidate is 73 which is not less than the current selected value of 32, so 73 is ignored.



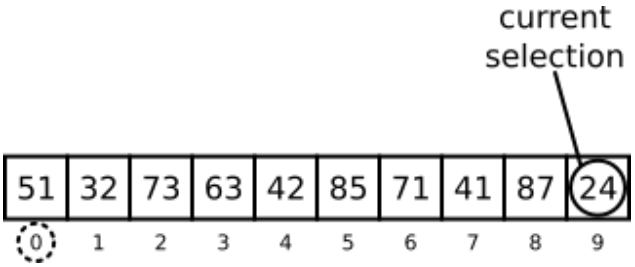
Selection Sort continues to compare candidate values including comparing 63 with the current selected value of 32, and because 63 is not less than the current selected value, 63 is ignored.



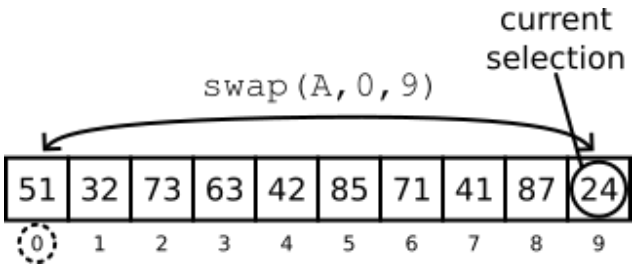
Selection Sort compares all remaining values to the current selected value but does not find a replacement until it reaches the last value in the array. 24 is compared to 32.



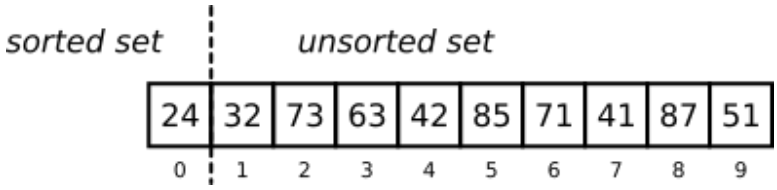
Because 24 is less than 32, 24 becomes the current selected value.



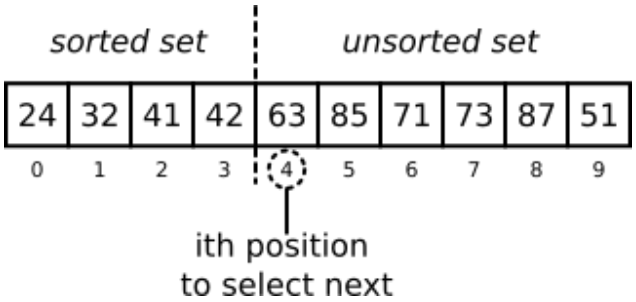
Because Selection Sort has reached the end of the set, there are no further comparisons to be made and the minimum value in the unsorted set must be the current selected value. The current selected value is only now swapped with the selected position.



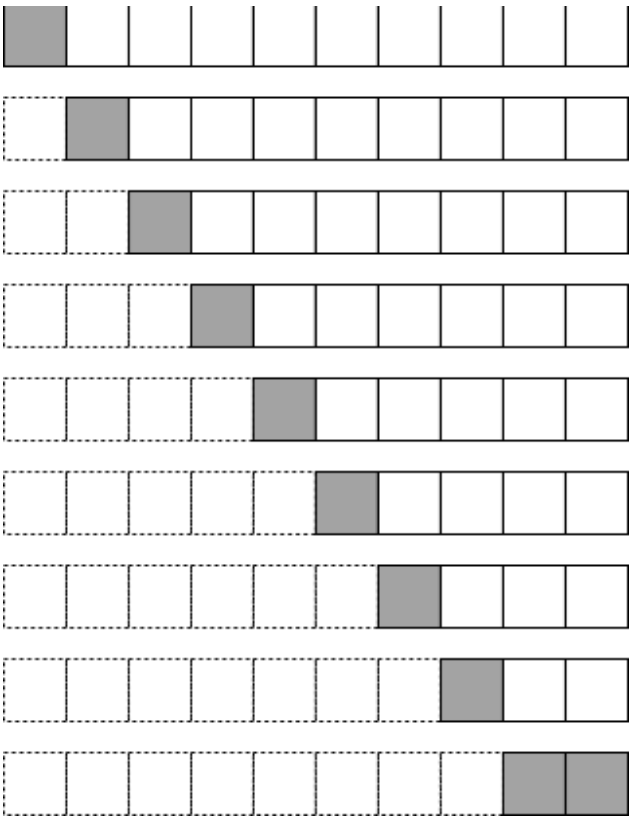
Once the swap is completed, the smallest value is guaranteed to be in the first position of the array. We can imagine a dividing line between the 1st and 2nd position where elements to the left of the division belong to a sorted subset and elements to the right of the division belong to the remaining unsorted subset.



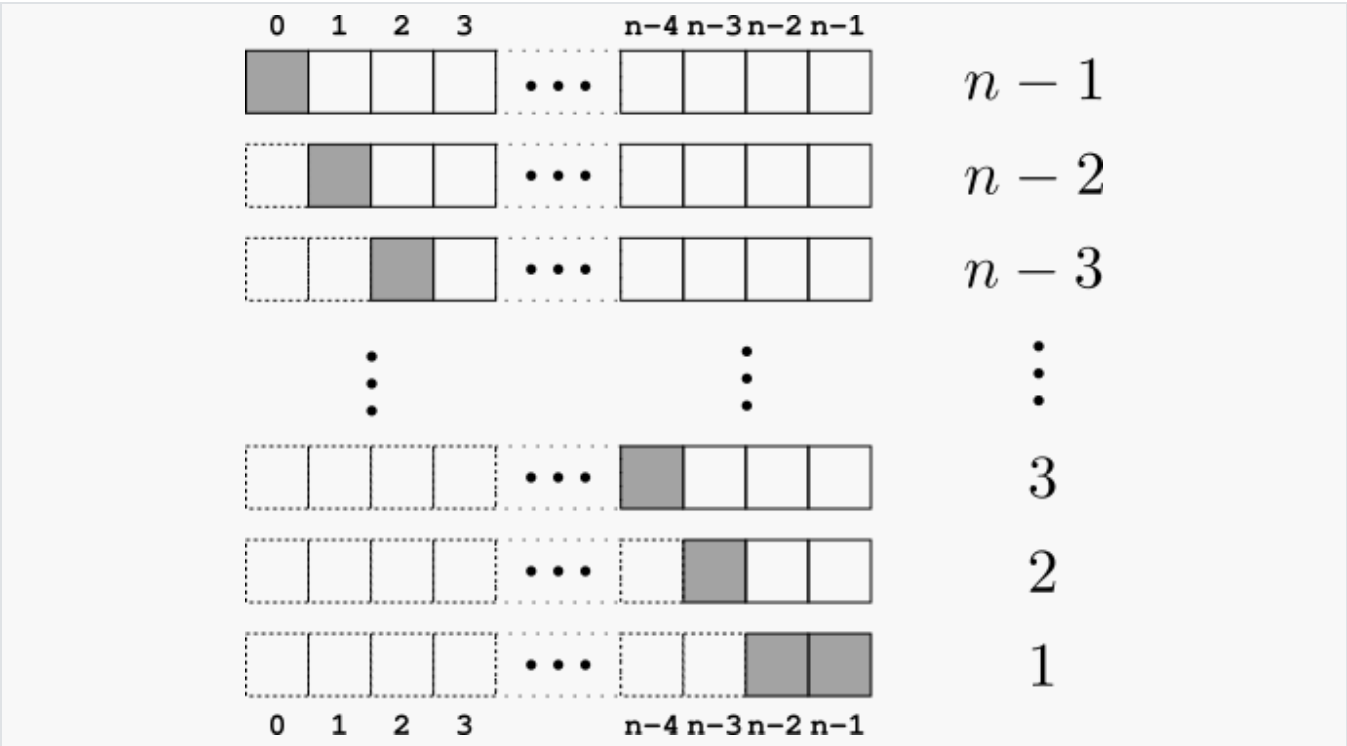
With each pass, Selection Sort is selecting the i-th smallest element of the overall set, but more simply, Selection Sort is selecting the smallest element of the unsorted set which should suggest that there is a recursive approach to this problem. In other words, the unsorted set is being reduced in size by one element with each pass and eventually the unsorted set will become a set of size zero.



Selection sort makes multiple passes through the set. The general result of a single pass through the data is that one element is placed into sorted order, the unsorted set shrinks by one element each pass, and the sorted set grows by one element. This pattern can be visualized in the following way where the gray box represents the sort position, solid outlined boxes represent members of the unsorted set, and dashed outlined boxes represent members of the sorted set. The general result is consistent up until the last pass where two elements are sorted which occurs because selecting and sorting one element of a two element set puts all elements in order.



The above diagram can be generalized into the following diagram so that we can do an inductive step in the mathematical analysis.



To the right is illustrated the number of comparisons performed in a pass. On the first pass, n-1 comparisons are needed to put the smallest element into its sorted position, on the second pass, n-2 comparisons are needed to put the second smallest element into its sorted position, on the second to last pass, two comparisons are needed to put the third largest element into its sorted position, and on the last pass, one comparison is needed to put the second largest and the largest elements into their sorted positions. From this model we can perform a formal mathematical analysis.

Mathematical Analysis

Where n is the size of the input. If we sum the values along the right side, we get the following expression that represents the total number of comparisons for Selection Sort:

$$(n-1)+(n-2)+(n-3)+\cdots+3+2+1$$

We know that there are a total of n-1 terms in this equation from the diagram above:

$$\underbrace{(n-1)+(n-2)+(n-3)+\cdots+3+2+1}_{n-1}$$

Assuming that n is an odd number, $n-1$ is an even number and the number of constants on the right of the expression equals the number of terms on the left of the expression:

$$\underbrace{(n\cancel{-1})+(n\cancel{-2})+(n\cancel{-3})+\cdots+\cancel{3}+\cancel{2}+\cancel{1}}_{n-1}$$

Assuming the n is an odd number is a just a convenience for us to reason this out. Since this is a Big O analysis we will be dropping coefficients and smaller terms, so it does not really matter whether n is odd or even other than to simplify our math model during the analysis.

Since cancellation has eliminated all constants, half of the terms in the expression are eliminated and the equation is reduced to a summation involving elements with only a value of n:

$$\underbrace{n+n+\cdots+n}_{\frac{n-1}{2}}$$

By the definition of multiplication, we can simplify the above expression into the following:

$$n\cdot\frac{n-1}{2}$$

Distributing n gives us the following equation:

$$\frac{1}{2}n^2-\frac{1}{2}n$$

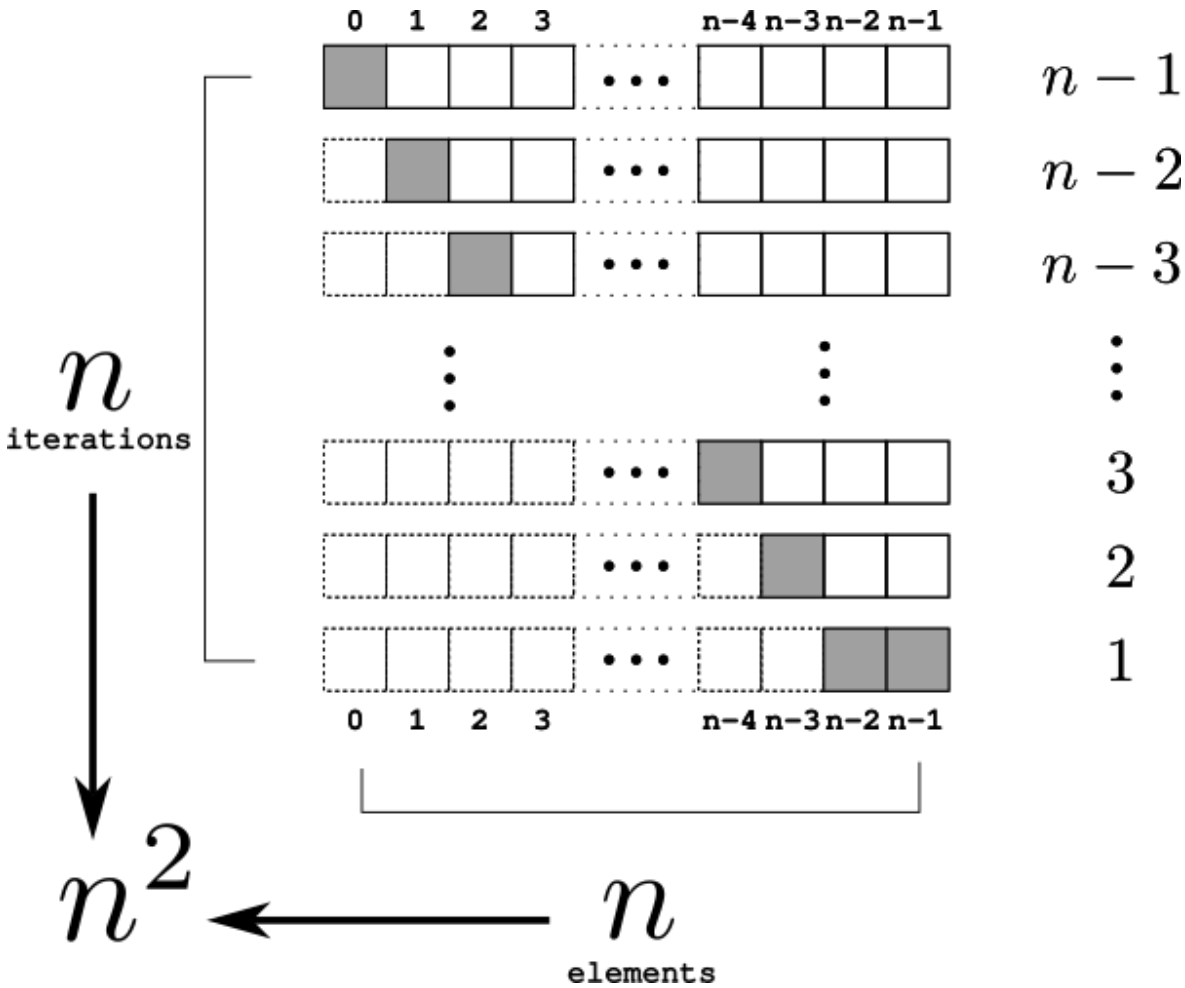
Since we are performing time complexity analysis, we can drop all coefficients and lesser terms to arrive at a big O estimation of $O(n^2)$.

The formal mathematical analysis is involved and requires a solid mathematical foundation to reach a correct conclusion. This might lead us to a search for another, easier way to analyze the data in the abstract without relying on extensive mathematical skills.

Graphical Analysis

One of the features of Big O is that it is an estimated and abstract form of analysis and it is not necessary to compute coefficients and constants for most general programming. The same analysis can be performed by looking at the dimensionality of the process which does not require the same depth of mathematical expertise.

Where n is the size of the input, the earlier diagram of can be generalized as Selection Sort performs approximately n passes (iterations) through the data and for each pass it performs approximately n comparisons among n elements. In other words, it performs n iterations and operates on n elements per iteration. From this we can estimate that it has a Big O of $O(n^2)$,



The above techniques may be useful, but algorithms are expressed in code, whether pseudocode or in a specific language. This suggests the need to develop the same analytic tools only more code centric.

Pseudocode Analysis

The pseudo code for Selection Sort can be expressed as the following:

```
Input : E is the input set and is considered unordered
Output : E is sorted in place, so E implicitly returns as the ordered set selectionsort(E):
    for i = 1 to length(E):
        selected = i
        for j = i+1 to length(E):
            if E[j] < E[selected]:
                selected = j
        swap(E[i], E[selected])
```

This raises the question of how Big O efficiency can be estimated from the code above. The answer is deceptively simple for Selection Sort, but this simplicity should never be assumed without careful consideration of an algorithm.

Where n is the size of the input, n is equal to `length(E)`. The maximum bound of the outer loop would result in n iterations and the maximum bound of the inner loop would result in n iterations. In other words, for a single iteration of the outer loop, the inner loop iterates on the order of n times and therefore for an approximate total of n iterations of the outer loop the innermost block of code is repeated approximately $n \cdot n$ times. This indicates that the algorithm is $O(n^2)$ in the worst case.

Selection Sort could be described as a brute force algorithm that uses a grand strategy of finding the smallest value in the unsorted set and putting it in the largest sorted position of the sorted set. This strategy comes at the price of an $O(n^2)$ in the worst case algorithm. Can Selection Sort ever do better than $O(n^2)$?

Best Case & Average Case

As it turns out, there are no possible optimizations or data configurations that allow Selection Sort to ever perform better than $O(n^2)$. In other words, Selection Sort's best case and average case equal its worst case and it performs as an $O(n^2)$ algorithm under all conditions. This is a weakness of its strategy as it can never know if it has selected the best candidate in any pass until it has performed comparisons with all other elements remaining in the unsorted set.

Are there algorithms that do not suffer from this limitation? It would be beneficial to have a sorting algorithm that even though it might be $O(n^2)$ in the worst case can perform better in the average or best case.

Stability

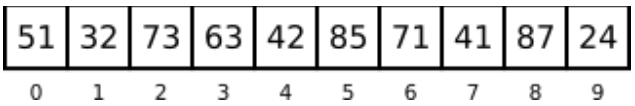
Selection sort is an unstable sorting algorithm

TODO: Illustrate

Bubble Sort

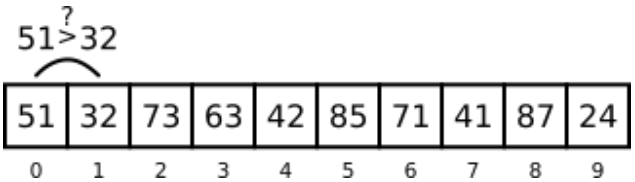
Bubble Sort like Selection Sort is a brute force sorting algorithm; however, Bubble Sort builds its strategy on a simple, localized technique involving neighbor comparison. In short, Bubble Sort iterates through the array, compares each pair of neighboring elements, and exchanges them if they are out of order. If this process is repeated frequently enough on all the data, the data will eventually reach a sorted state.

Given an array with the following data:

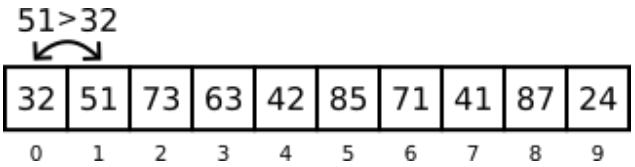


This initial array again represents a completely unsorted set on which we wish to perform Bubble Sort and again suppose this array is to be sorted in ascending numerical order.

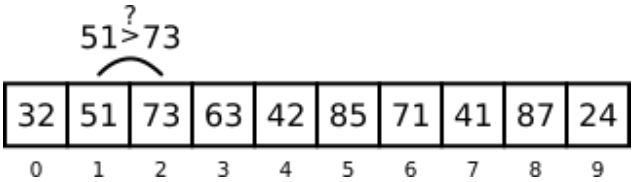
Bubble Sort starts at the beginning of an array and compares the first two neighboring elements, *i.e.* elements at indices 0 and 1.



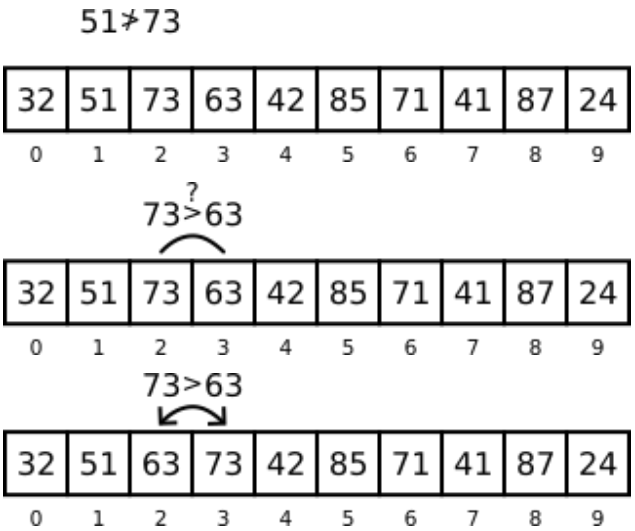
In the case of the first pair of neighbors, 51 is greater than 32, so this pair is out of order and this pair of neighbors need to swap positions to locally order the two which will increase the overall order of the set.



After the first pair of neighbors are compared and if necessary swapped, the second pair of neighbors is compared. The second pair is composed of the following element (the element that landed on the right side at the end of the comparison and potential swap) from the previous comparison and the element to its right, *i.e.* elements at indices 1 and 2.



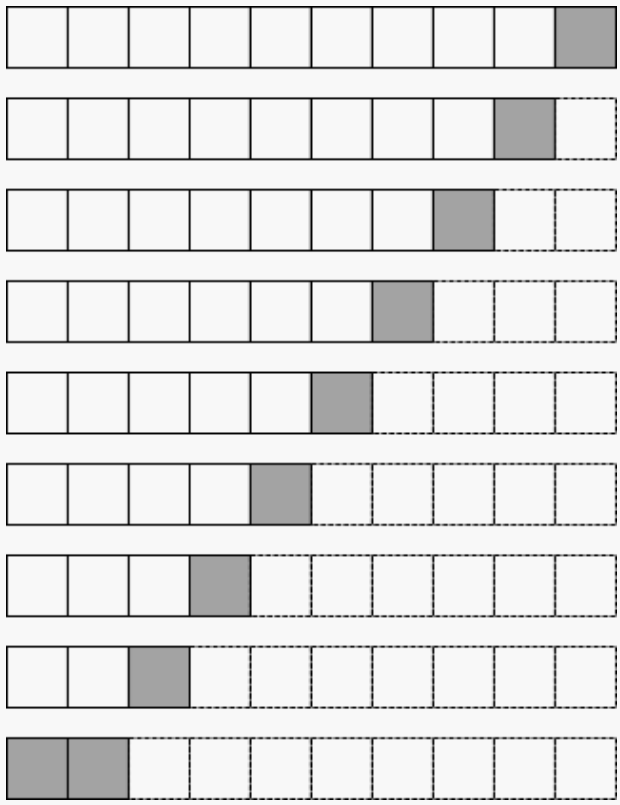
The second pair of neighboring elements is already in order.



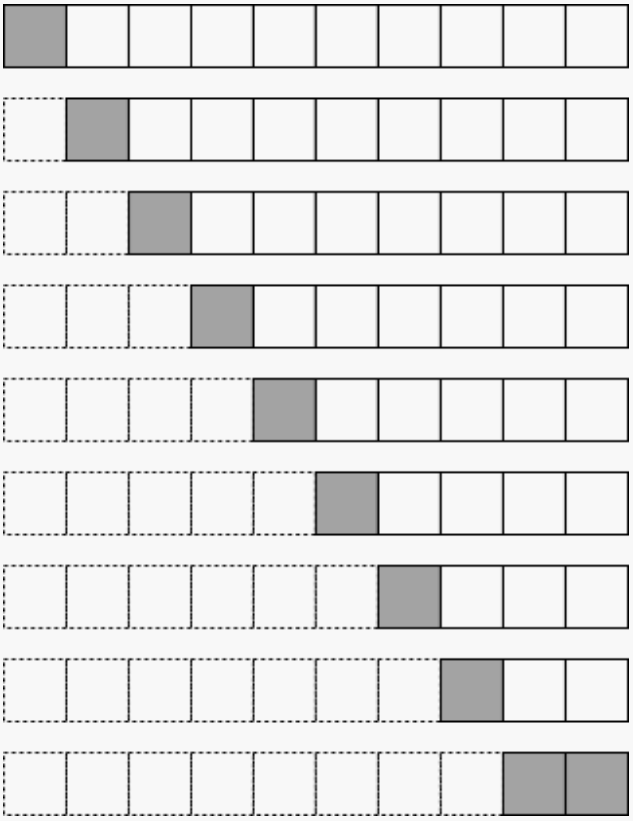
At the end of the first pass the array has the following order.

32	51	63	42	73	71	41	85	24	87
0	1	2	3	4	5	6	7	8	9

Note that the largest number has landed in the rightmost spot and at the completion of each subsequent pass at least one element, *i.e.* the largest remaining element in the unsorted set, is sorted into the sorted set. The block diagram for Bubble Sort tells an almost identical story to that of Selection Sort except that the diagram for Bubble Sort is the mirror of that for Selection Sort.



Block diagram illustrating Bubble Sort in the worst case.



Block diagram illustrating Selection Sort in the worst case.

For this illustration and implementation of Bubble Sort, one sorted set element is guaranteed to be sorted each pass and the sorted set aggregates to the right side of the array while the unsorted set is reduced from the left side of the array. As before, the gray element illustrates the position of the element that is guaranteed to be sorted at the end of each pass.

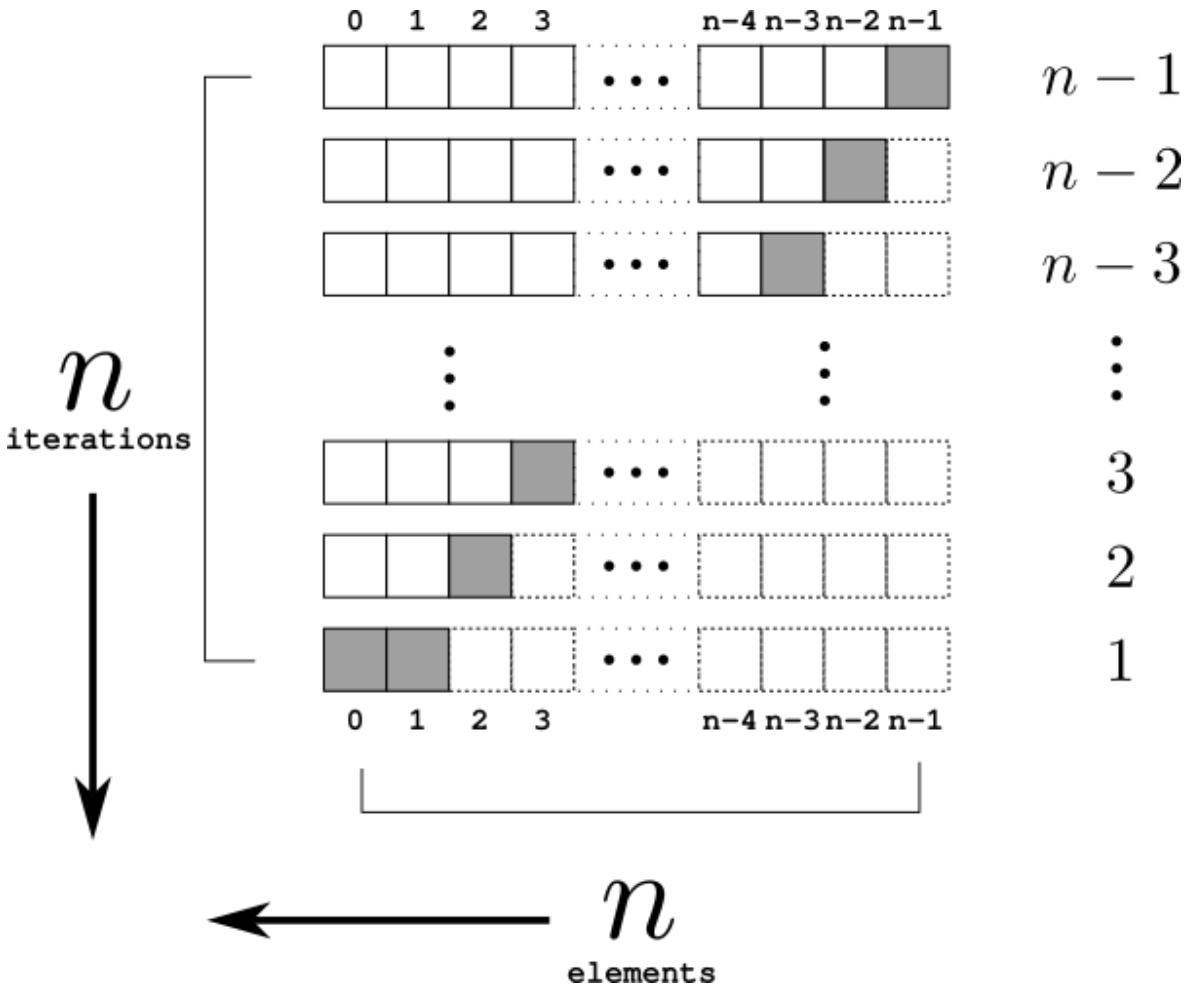
Bubble Sort is built on the idea that increasing the local order of a pair of elements increases the order of the overall set and if this process is repeated enough times, the entire set will become ordered. The overall efficiency depends on what is “enough times”. The block diagram illustrates that in the worst case Bubble Sort has the same efficiency as Selection Sort.

Mathematical Analysis

As the block diagram above suggests, the mathematical worst case proof for Bubble sort is identical to the proof for Selection Sort. Refer to the section on Selection Sort for a detailed analysis.

Graphical Analysis

The similarities between Bubble Sort and Selection Sort block diagrams suggests the same inductive technique is used to estimate the efficiency of Bubble Sort in a dimensional analysis. The generalized diagram for Bubble Sort is also a reflection of the Selection Sort diagram.



From this we can estimate that Bubble Sort is Big O of $O(n^2)$ where n is the size of the input.

Pseudocode Analysis

The pseudo code for Bubble Sort can be expressed as the following:

```
Input : E is the input set and is considered unsorted
Output : E is sorted in place, so E implicitly returns as the sorted set
n: the size of E
ei: the ith element of e
ei+1: the neighboring element to the right of e
bubblesort(E):
    for i = 0 to n-1:
        for j = 0 to n-i:
            if ej > ej+1:
                swap(ej+1, ej)
```

As in Selection Sort, the maximum bound of the outer loop would result in n iterations and the maximum bound of the inner loop would result in n iterations which indicates that the algorithm is $O(n^2)$ in the worst case where n is the size of the input.

Bubble Sort could also be described as a brute force algorithm that uses a relative, neighbor comparison strategy to produce a sorted set. This strategy comes at the price of an $O(n^2)$ in the worst case algorithm and also raises the question of whether or not Bubble Sort is $O(n^2)$ in the best and average cases.

Best Case & Average Case

While Bubble Sort has the same efficiency in the worst case as Selection Sort, there may be an objectively supportable argument that Bubble Sort is a better algorithm than Selection Sort depending on Bubble Sort's performance in the best or average cases. The original algorithm proposed above struggles with the same deficiency that existed in Selection Sort, *i.e.* there is no way to exit early; however, not all inputs are the same.

Consider how long Selection Sort will take on an ordered set:

24	42	51	63	71	73	85	87	32	41
0	1	2	3	4	5	6	7	8	9

With Selection Sort, there is no way to determine if the set is already ordered, much less a way to exit the process in an operation in the middle of the process results in an ordered set. Selection Sort is designed to put the i-th best element in the i-th position and the i-th best can only be operated on when all proceeding positions have been validated through selection. Selection Sort is a more holistic approach in terms of ordering the entire array and is a distinct contrast from the more relative, neighborly approach of Bubble Sort. Since these approaches are so different, it raises the question of whether or not it is possible to alter Bubble Sort in a way that allows Bubble Sort to exit the process early if the set is ordered.

Consider this question, what is indicated in Bubble Sort if it completes a pass and a swap is never performed?

Remember that if a pair of neighbors are out of order, Bubble Sort swaps them when it encounters them during a pass. If we complete an entire Bubble Sort pass through the data and no swap is made, the data must already be in order. Any subsequent passes through the data will not alter the order of the data, so they are wasted operations. Armed with this information, let's improve the initial Bubble Sort algorithm to detect when a swap is not made during a pass and exit the algorithm at that point.

```
Input : E is the input set and is considered unsorted
```

```
Output : E is sorted in place, so E implicitly returns as the sorted set
n: the size of E
ei: the ith element of e
ei+1: the neighboring element to the right of e
bubblesort(E):
    for i = 0 to n-1:
        swapped = false             // set swapped flag to false at start of pass
        for j = 0 to n-i:
            if ei > ei+1:
                swap(ei+1, ei)
                swapped = true      // flip swapped flag to true if swap is performed
        if not swapped:             // after completing pass, check if swap NOT performed
            done                    // if no swap, then data is in order, get out
```

In the revised approach to Bubble Sort, a boolean "flag", swapped, is used to determine if a swap is performed during a pass. The flag is set to false at the beginning of a pass and if a swap is performed during the pass, the flag is switched to true to indicate that a swap was performed during this current pass. It does not matter how many swaps are performed, it only matters if at least one swap is performed. If no swap is performed during a complete pass through the data, no further swaps will be performed in any subsequent passes and the data must be in order. If the data is in order, there is no need to continue the process.

Stability

Bubble sort is a stable sorting algorithm

TODO: Illustrate

Insertion Sort

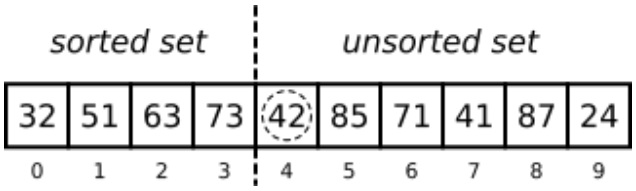
Insertion Sort uses a similar neighbor comparison strategy as Bubble Sort, but it makes use of an observation about the sorted set to minimize the number of unnecessary comparisons and swaps that may occur in Bubble Sort.

Bubble Sort and Insertion Sort are similar, but there is an important and subtle difference in their design principle. In a single pass, Bubble Sort guarantees the extreme-most unsorted element propagates to the end of the sorted set where it joins the sorted set in its correct position. In a single pass, Insertion Sort guarantees that the current element is inserted into the sorted set at its correct sorted position relative to the other elements in the sorted set. In other words, Insertion Sort finds the correct position for the next element in the sorted set with respect to the elements in the sorted set while Bubble Sort sorts the i-th smallest or largest element into its absolute position in the sorted set. In this respect, while Bubble Sort uses the neighbor swapping strategy the overall behavior is more similar to Selection Sort than it is to Insertion Sort.

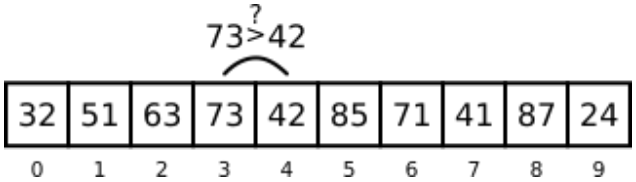
Given an array with the following data:

51	32	73	63	42	85	71	41	87	24
0	1	2	3	4	5	6	7	8	9

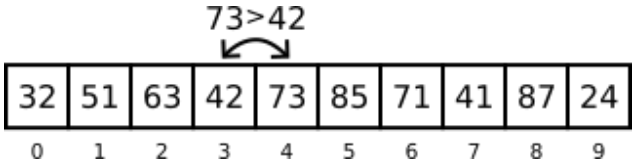
For Insertion Sort, it is difficult to discern what is going on from the initial few passes, so the following examples start at sorting the element at index 4 into its correct position in the sorted set. At the beginning of this pass the array has the following state:



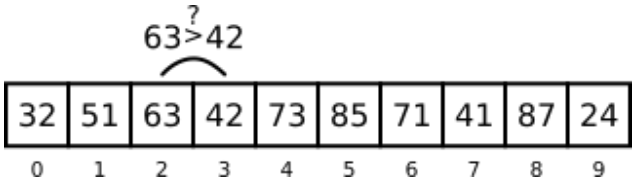
The sort element, as indicated by the dashed circle, with the value 42 is compared with the element to its left, *i.e.* the rightmost (largest) element in the sorted set here at the first comparison, 72.



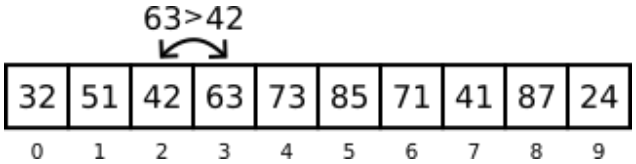
If the sort element is less than the element to its left, the two values are swapped and the process repeats as is the case for this example, *i.e.* 73 > 42 so they are out of order and must be swapped.



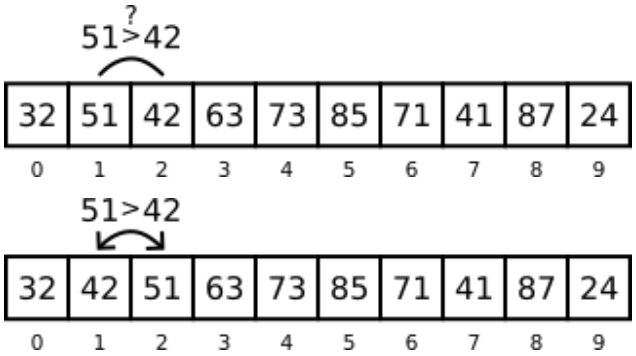
The process repeats itself where the sort element is compared to the next neighboring element to its left.



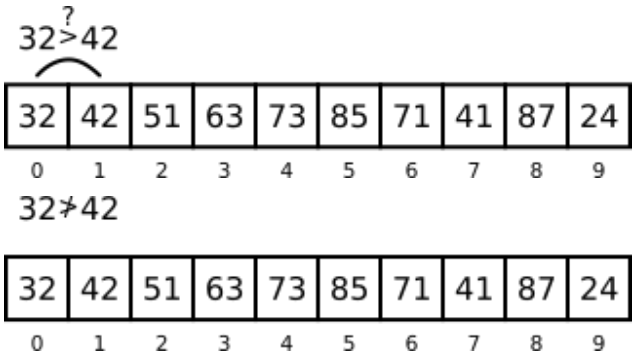
Again, if 42 is less than the neighboring element, they are swapped.



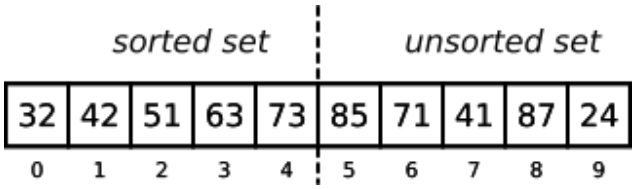
This process is repeated as long as 42 is less than subsequent neighboring elements.



The process ends if the next neighboring element to its left is not less than the sort element. Since this propagation occurs within the ordered set, we are assured that if the next neighboring element to the left is not less than the sort element, then all elements to the left of the sort element are not less than the sort element and the sort element has reached its sorted position.

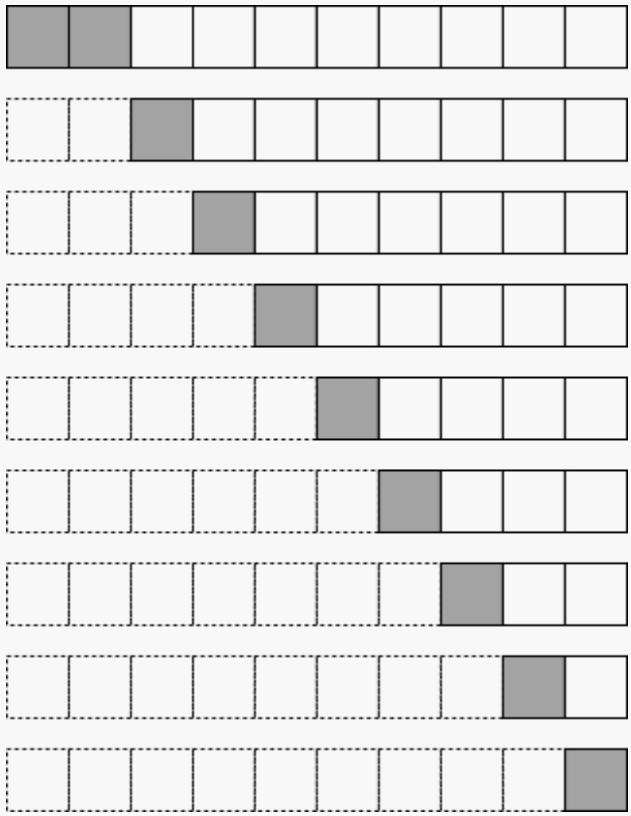


At the end of the pass for the i-th element, it is guaranteed to be in the correct sorted position within those elements that have joined the sorted set and the unsorted set is reduced by one element. Insertion Sort is ready to perform the same process on the next unsorted element.

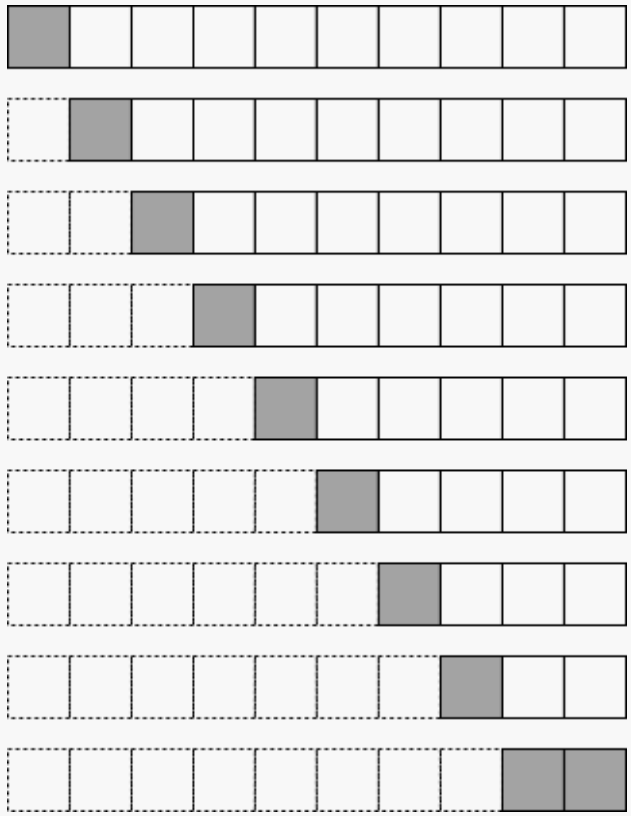


Graphical Analysis

The overall Insertion Sort process resembles the same set groupings that emerge from Selection Sort which suggests Insertion Sort has the same worst case performance as Selection Sort.



Block diagram illustrating Insertion Sort



Block diagram illustrating Selection Sort

But consider when Insertion Sort produces its worst case performance. We found that Selection Sort's best case and average case are the same as Selection Sort's worst case, because due to its strategy, the process has no way to exit the process early and the same number of comparisons will be performed regardless of the ordering of the input data. In contrast, we found that Bubble Sort has an optimization that allows it to exit early if there are no swaps, and from this emerges a best case where if the input data is already ordered Bubble Sort will make only one pass which has $O(n)$ efficiency.

This raises the question of whether Insertion Sort similarly has advantageous best and average cases. In fact, Insertion Sort only performs at its worst case if the input data is sorted in reverse order. Given a reverse order input, on each pass through the data, Insertion Sort must move the current sort element all the way to the beginning of the sorted set which results in the same number of total iterations as Selection Sort; however, Insertion Sort is actually worse than Selection Sort in this specific case because a comparison + swap for each iteration is more expensive than just a comparison.

Pseudocode

```
Input : E is the input set and is considered unsorted
Output : E is sorted in place, so E implicitly returns as the sorted set
n: the size of E
ej: the jth element of e
ej+1: the neighboring element to the left of e
insertionsort(E):
    for i = 1 to n-1:
        for j = i to 1:
            if ej-1 > ej:
                swap(ei+1, ei)
            else:
                break
```

Best Case & Average Case

The worst case for Insertion Sort is established above, but to really understand that Insertion Sort was an advancement over both Selection Sort and Bubble sort, we must examine the best and average cases.

The best case for Bubble Sort was established above to be O(n), so if Insertion Sort needs to be equivalent, and it is. If the data is already sorted, then the outer loop will loop over all elements as normal, but only a single operation will be performed in the inner loop each pass as the inner loop will break on the first comparison which will result in a no swap.

But what about the average case for Insertion Sort? We did not even explore Bubble Sort's average case because it is extremely difficult to identify given randomized data and we generally assume it to tend to Bubble Sort's worst case. For Insertion Sort to be an improvement on Bubble Sort, Insertion Sort's average case needs to be better than Bubble Sort's worst case, so let's see if we can prove it.

To address the average case, we will assume that half the time we have to move an element from its initial position all the way to the left-most position in the sorted set and half the time we don't have to move an element at all because it is already in its sorted position. This means all of the terms in the original proof are divided by two and we proceed through the proof as normal.

$$\underbrace{\frac{1}{2}+\frac{2}{2}+\frac{3}{2}+\cdots+\frac{n-3}{2}+\frac{n-2}{2}+\frac{n-1}{2}}_{n-1}$$

$$\underbrace{\frac{1}{2}+\frac{2}{2}+\frac{3}{2}+\cdots+(\frac{n}{2}-\frac{3}{2})+(\frac{n}{2}-\frac{2}{2})+(\frac{n}{2}-\frac{1}{2})}_{n-1}$$

As we did in the Selection Sort analysis, we assume that n is an odd number and $n-1$ is an even number, so the number of constants on the right of the expression equals the number of terms on the left of the expression allowing us to cancel all of the constant terms:

$$\underbrace{\cancel{\frac{1}{2}} + \cancel{\frac{2}{2}} + \cancel{\frac{3}{2}} + \cdots + (\frac{n}{2} \cancel{- \frac{3}{2}}) + (\frac{n}{2} \cancel{- \frac{2}{2}}) + (\frac{n}{2} - \cancel{- \frac{1}{2}})}_{n-1}$$

Similarly, cancellation eliminated all constants, so half of the terms in the expression are eliminated and the equation is reduced to a summation involving elements with only a value of $\frac{n}{2}$:

$$\underbrace{\frac{n}{2} + \frac{n}{2} + \cdots + \frac{n}{2}}_{\frac{n-1}{2}}$$

Again, using the definition of multiplication, we can simplify the above expression into the following:

$$\frac{n}{2} \cdot \frac{n-1}{2}$$

Distributing $\frac{n}{2}$ gives us the following equation:

$$\frac{1}{4}n^2 - \frac{1}{4}n$$

So, our average case time complexity for Selection Sort is $O(n^2)$ which is no different than our worst case time complexity.

However recall the details of the math we used to derive the average case for both selection sort and insertion sort.

	Selection Sort	**Insertion Sort**
Avg Number of Comparisions	$\frac{1}{2}n^2 - \frac{1}{2}n$	$\frac{1}{4}n^2 - \frac{1}{4}n$
Avg Case Time Complexity	$O(n^2)$	$O(n^2)$

From the above selection sort generally has the same time complexity as insertion sort, but from a practical standpoint, the average number of comparisons for insertion sort is significantly less. In other words, they will both grow quadratically with the size of the input, but objectively, insertion sort will almost always require less comparisons than selection sort.

n	**Avg Selection Sort Comparisons**	**Avg Insertion Sort Comparisons**
10	45	23
100	4,550	2,475
1000	499,500	249,750

Stability

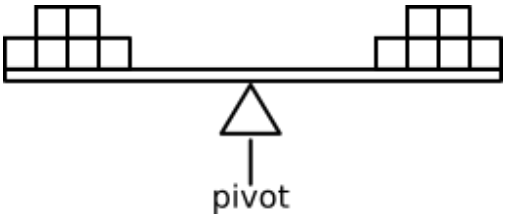
Insertion sort is a stable sorting algorithm

TODO: Illustrate

Quicksort

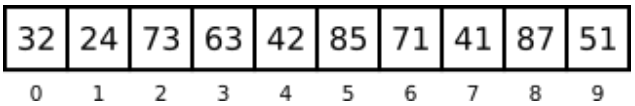
Quicksort operates in a way unlike all the other sorting algorithms we have so far studied. Instead of using a brute force strategy, it uses a divide and conquer strategy to sort the data in the input set.

Quicksort is built on the idea of a pivot which acts much like the pivot on which a see-saw balances. In a perfectly balanced see-saw, half the weight is distributed on one side which is counterbalanced by half the weight on the other side.

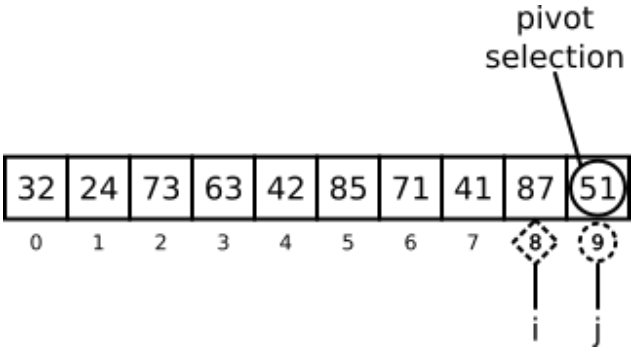


For Quicksort, the first step is selecting a pivot, followed by moving any elements that are larger than the pivot to the right side of the array and moving any elements that are smaller than the pivot to the left side of the array, and finally swapping the pivot into the pivot's correct location in the array. This process is then repeated on the subset to the right of the pivot and on the subset to the left of the pivot. The process concludes when the subsets are empty.

Given an array with the following data:

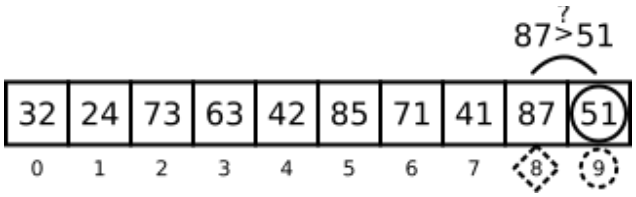


This example uses a naive pivot selection where the leftmost element is arbitrarily chosen as the pivot. In addition to tracking the pivot location, the pivot algorithm also tracks two indices: i indexes a loop through the set and j tracks the index that divides the left and right sets.

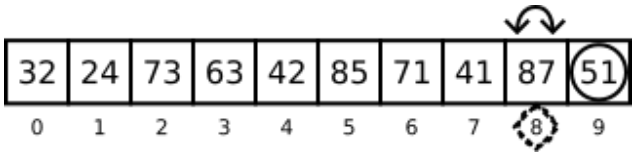


i begins at the index to the left of the pivot and j at the pivot index.

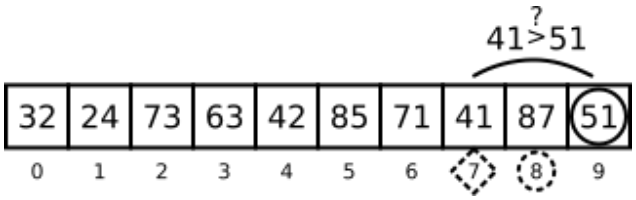
At each iteration of the loop, the i-th element is compared with the pivot element. If the i-th element is greater than the pivot element, j is moved to the left one position and the i-th and j-th element are swapped.



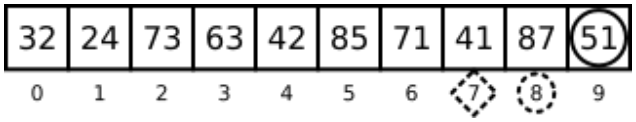
This presents a bit of an odd case in the example where the first element is swapped with itself, but this cost is marginal and eliminates any need for the algorithm to have additional branches.



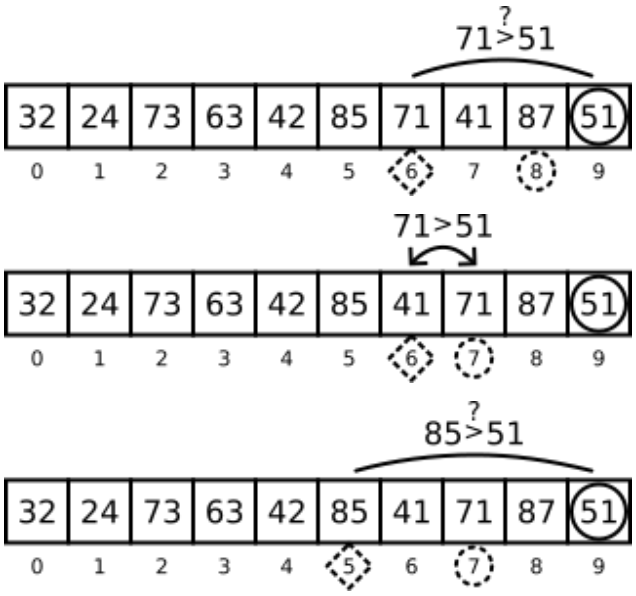
The pivoting process proceeds and compares the next element with the pivot

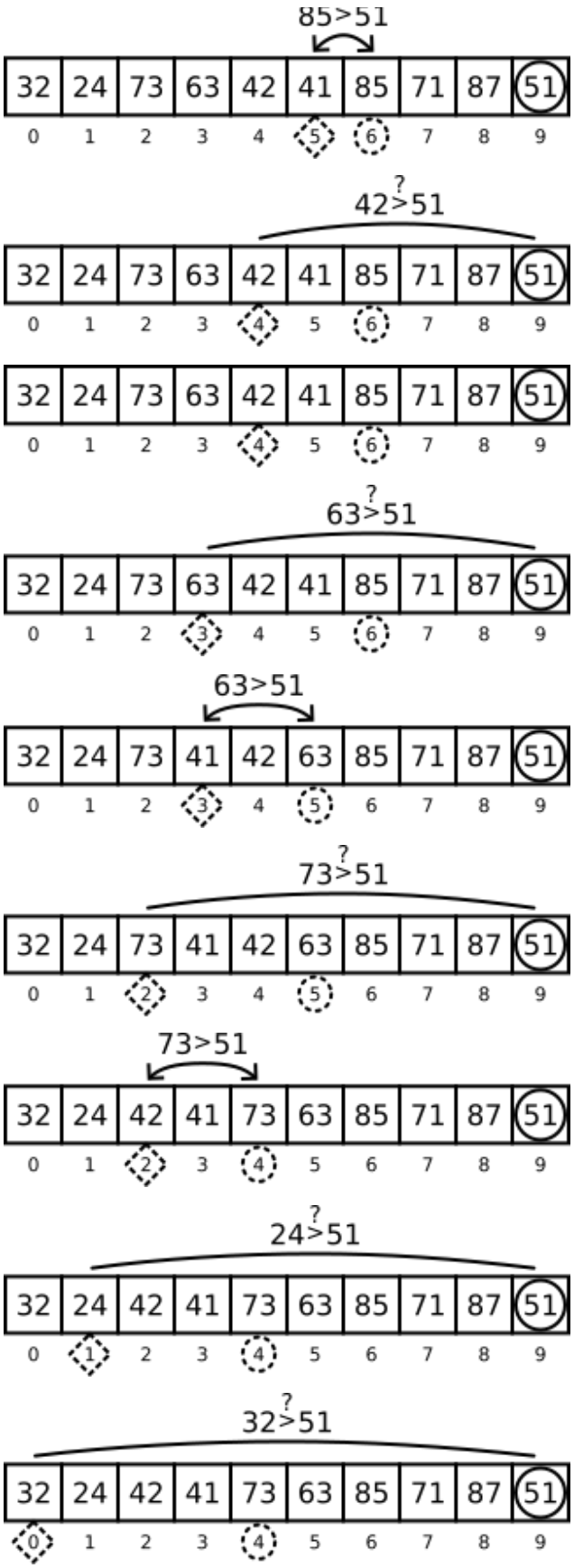


In this case, the next element is less than the pivot, so no swap is performed.

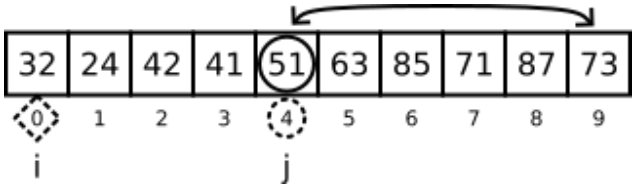


Because no swap was performed in the previous step, the two indices, i and j, will start to diverge and the left and right sets will be more apparent, *i.e.* elements between i and j are in the subset that is smaller than the pivot element and elements at j up to but excluding the pivot index are in the subset that is larger than the pivot element. The process repeats until the leftmost end of the array is reached.

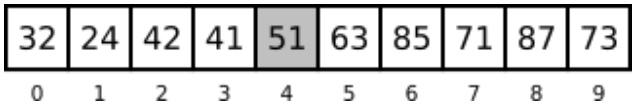



85>51
32 24 73 63 42 41 85 71 87 51
0 1 2 3 4 5 6 7 8 9
42?>51
32 24 73 63 42 41 85 71 87 51
0 1 2 3 4 5 6 7 8 9
32 24 73 63 42 41 85 71 87 51
0 1 2 3 4 5 6 7 8 9
63?>51
32 24 73 63 42 41 85 71 87 51
0 1 2 3 4 5 6 7 8 9
63>51
32 24 73 41 42 63 85 71 87 51
0 1 2 3 4 5 6 7 8 9
73?>51
32 24 73 41 42 63 85 71 87 51
0 1 2 3 4 5 6 7 8 9
73>51
32 24 42 41 73 63 85 71 87 51
0 1 2 3 4 5 6 7 8 9
24?>51
32 24 42 41 73 63 85 71 87 51
0 1 2 3 4 5 6 7 8 9
32?>51
32 24 42 41 73 63 85 71 87 51
0 1 2 3 4 5 6 7 8 9

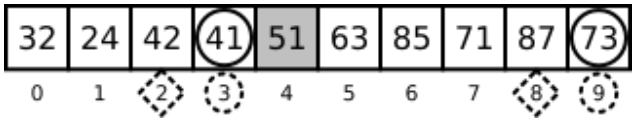
When the last index is processed, the pivot element is swapped with the element at index j, i.e. the index of the leading element in the right subset, which puts the pivot element into its sorted position.



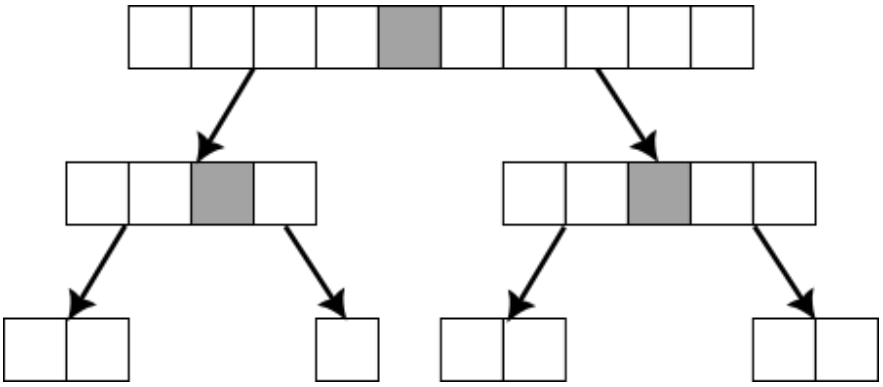
All elements to the left of the pivot element are less than or equal to the pivot element and all elements to the right of the pivot element are greater than the pivot element. While the left and right subsets may not be sorted yet, the pivot is in its correct, sorted position within the overall set.



The process is then repeated on the left and right subsets.



This second pass will result in two elements being placed into their sorted positions with each subsequent pass resulting in twice the number of previous pivots being sorted into their correct location at the cost of a single pass through the data assuming that pivot selection is ideal for each subset.



Mathematical Analysis

Let us assume that pivot selection is ideal, meaning that pivot selection always produces the median (middle-most) element in the set using a constant time algorithm. If pivot selection is optimal, then the mathematical analysis is as follows.

At each pass, each subsets is divided in half until each subset consists of a single element.

The first pass the algorithm processes the entire set performing n comparisons:

$$n$$

The second pass the algorithm also processes the entire set performing a total of n comparisons on two subsets; however, assuming the pivot selection perfectly bisects the set, then there are $\frac{n}{2}$ elements in two subsets:

$$\underbrace{\frac{n}{2} + \frac{n}{2}}_{2}$$

Each pass continues to bisect each subset producing twice as many subsets, but performing half as many comparisons on each subset. For a given pass though the data, no more than n comparisons are made:

$$\underbrace{\frac{n}{4} + \frac{n}{4} + \frac{n}{4} + \frac{n}{4}}_{4}$$

The process continues until the subsets contain only one element:

$$\underbrace{1 + 1 + \cdots + 1}_{n}$$

At which point, the open remaining question x is how many times times was it possible to subdivide each subset until each subset contained only one element.

$$\underbrace{(n) + (\frac{n}{2} + \frac{n}{2}) + (\frac{n}{4} + \frac{n}{4} + \frac{n}{4} + \frac{n}{4}) + \cdots + (\underbrace{1 + 1 + \cdots + 1}_{n})}_{x}$$

Based on our earlier discussion of logarithms, the base 2 logarithm predicts the maximum number of times a set can be repeatedly divided in half. In other words:

$$x = \log_2 n$$

Giving us the following equation for the total number of comparisons:

$$\underbrace{(n) + (\frac{n}{2} + \frac{n}{2}) + (\frac{n}{4} + \frac{n}{4} + \frac{n}{4} + \frac{n}{4}) + \cdots + (\underbrace{1 + 1 + \cdots + 1}_{n})}_{\log_2 n}$$

Simplifying each of the parenthetical subequations yields:

$$\underbrace{n + n + \cdots + n}_{\log_2 n}$$

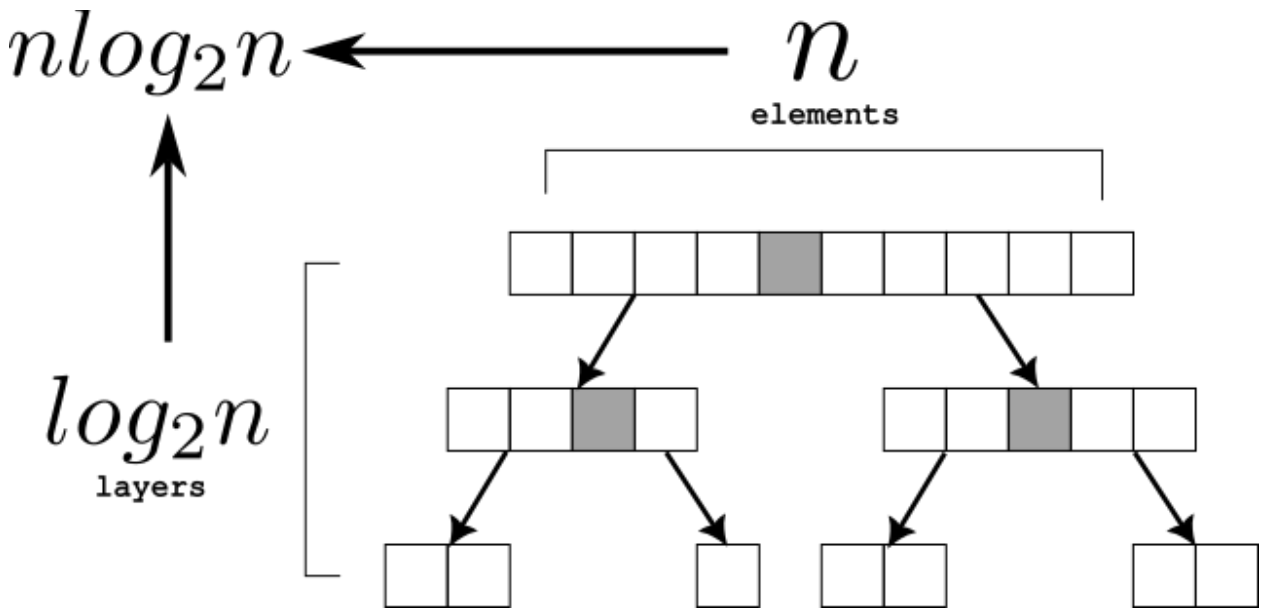
Again, applying the definition of multiplication simplifies to:

$$n \log_2 n$$

In other words, given an ideal pivot selection algorithm, Quicksort's efficiency is $O(n \log_2 n)$ or loglinear time complexity.

Graphical Analysis

Again, we can perform a dimensional analysis to clearly see the efficiency is $O(n \log_2 n)$.



Pseudocode Analysis

Since this is a recursive algorithm, analysis becomes more difficult and in this case, it is much easier to model the efficiency using one of the other techniques. The risk here is seeing two recursive calls per call to the function and assuming that the algorithm is exponential without recognizing that the size of the sets are diminishing by half each time.

```
Input : E is the input set and is considered unsorted
Output : E is sorted in place, so E implicitly returns as the sorted set
left : the leftmost position in the unordered set
right : the leftmost position in the unordered set
p : the pivot position
quicksort(E, left, right) :
    if left < right : sorted
        p ← pivot(E) where E defined by E[left, right]
        quicksort(E, right, p-1)
        quicksort(E, p+1, left)

pivot(E[left, right]) : E[left,pivot-1], E[pivot+1,right]
    if left = right :
        no remaining set
    pivot = right
    for ei in E[right, left]
        if epivot < ei
            pivot = pivot - 1
            swap(ei, epivot)

    swap(epivot, eright)
    return pivot
```

Best Case, Average Case, and Worst Case

Pivot selection is critical to the overall efficiency of Quicksort. What we analyzed above is actually the best case for quicksort with naive pivot selection. Pivot selection can become more robust and guarantee average case efficiency at the least if we just sampled three elements from the array during pivot selection and chose the median. With naive pivot selection average case performance is actually somewhere between $O(n \log_2 n)$ and $O(n^2)$ suggesting that it is really an $O(n^2)$ algorithm. However, Quicksort will generally perform closer to its best case if data is sufficiently randomized, so whenever performing a Quicksort analysis, you must consider the nature of the data.

Average case emerges because data is not likely to be perfectly balanced.

Degenerate Case

The degenerate case represents a specific situation where the expected behavior of the algorithm simply breaks down and performs extraordinarily poorly. We could argue this case arose in Insertion Sort with the reverse order input, but we are intentionally and somewhat arbitrarily introducing this worst-most case and making it distinct from the worst case here in Quicksort because students at this level have a tendency to assume this degenerate case is common enough to immediately conclude and argue that Quicksort generally performs at $O(n^2)$ when it does not if the pivot selection algorithm is slightly improved. Quicksort only performs at this worst-most case when the data is in order and the incredibly naive pivot selection algorithm proposed here is used. If we make a marginal improvement to pivot selection, the degenerate case disappears even with ordered data and Quicksort's worst case is somewhere between $O(n \log_2 n)$ and $O(n^2)$ but closer to $O(n \log_2 n)$ than to $O(n^2)$.

The degenerate case emerges when the data is already sorted and naive pivot selection is used. In this case, all elements are on one side of the pivot with each pass of quicksort. This means the block diagram generated resembles selection sort because only one element is sorted with each pass.

Cases that assume typical behavior will be described as having randomized input data and if the degenerate case is explicitly referenced, assume data is not random and has an order that triggers the worst-most case.

Stability

Quicksort is an unstable sorting algorithm. Instability can occur when a value with duplicates is selected as a pivot.

TODO: Illustrate

Visualization

https://www.cs.usfca.edu/~galles/visualization/ComparisonSort.html

Other Sorts

With the high variability in performance and list of special cases to consider for Quicksort, it is no wonder that further research was conducted into sort algorithms after Quicksort was published. Since this is an introductory course in algorithms, we will end our required discussion here but it is not the end of the sorting story. Additional sorts listed below are mentioned to ensure the breadth of the topic is illustrated and to encourage further study.

Merge Sort

Quicksort orders data while breaking the array down into constituent subsets but an alternative strategy is to break down the array into individual elements and then reassemble it in sorted order as we build it back up. This build it back up strategy in sorted order is the concept behind Merge Sort.

Merge Sort has better worst case performance than Quicksort, in fact it is $O(n \log_2 n)$ under all cases, and it is generally the best of the simple sorts to implement in terms of efficiency; however, it is difficult to visualize and understand its process for an introductory algorithms course. If you are interviewing for a programming position, it is important to familiarize yourself with this sort and to be prepared to implement it.

Review

1. What is the pass-by-value policy and how does it relate to arguments passed as parameters to functions?
2. Define the swap algorithm.
3. Why does pass-by-value cause a swap function that has primitive parameters to fail? Where in memory would the swap occur? Why does this lead to a failure in a swap function implementation?
4. Why can pass-by-value be bypassed by passing a reference to an object? Where are objects stored in memory and why is this location not governed by pass-by-value?
5. Summarize the strategy for Selection Sort
6. Summarize the strategy for Bubble Sort
7. Summarize the strategy for Insertion Sort
8. Summarize the strategy for Quicksort
9. Diagram the behavior of Selection Sort
10. Diagram the behavior of Bubble Sort
11. Diagram the behavior of Insertion Sort
12. Diagram the behavior of Quicksort
13. Define Pseudo-code for Selection Sort
14. Define Pseudo-code for Bubble Sort
15. Define Pseudo-code for Insertion Sort

16. Justify why the worst case, average case, and best case efficiencies for Selection Sort are always the same.
17. Justify why the best case for Bubble Sort is linear time complexity while its worst case is quadratic time complexity
18. Justify why Insertion Sort's average time complexity is better than Selection Sort's average time complexity.
19. Analyze the efficiency for Selection Sort.
20. Analyze the efficiency for Bubble Sort.
21. Analyze the efficiency for Insertion Sort.
22. Analyze the efficiency of Quicksort assuming the input is sufficiently randomized in the worst case.
23. Analyze the efficiency of Quicksort assuming the input is ordered. In other words, analyze the degenerate worst case efficiency for Quicksort.
24. Why does pivot selection determine the efficiency of Quicksort?
25. Given randomized input data, which sort algorithm among those we studied is the best choice? Justify your answer.
26. Given ordered input data, which sort algorithm among those we studied is the best choice? Justify your answer.
27. Given a system with extremely minimal memory and unordered data input of less than a million values, which sort is the best choice? Justify your answer.
28. Given a system with extensive amounts of memory and unordered data input on the order of a trillions of values, which sort is the best choice? Justify your answer.
29. Why do we sort?